

PROCESADORES DE LENGUAJE

ENTREGA TERCERA Y CUARTA SESIÓN



Nombre	NIA	Correo	Grupo	Equipo
Jorge López Alonso	100495876	100495876@alumnos.uc3m.es	85	509
Raoul Vlad Ivaszuk Ivaszuk	100508621	10508621@alumnos.uc3m.es	85	509

ÍNDICE

3.3.1 Cuestiones Resueltas de la Sesión.....	2
3.1 Adapta la gramática para que las expresiones encerradas con paréntesis puedan ir precedidas de un signo negativo (o positivo). Indicar sólo las producciones nuevas necesarias. Se recomienda probar la solución con bison. ¿Funciona para la expresión $-(-3)$?.....	2
3.2 Adapta la gramática para reconocer números reales (ejemplos: 00.33, 0.33, .33, 100, 0., 100.33). Representa las nuevas producciones de la gramática en BNF y en forma de Diagrama Sintáctico. No se pide ni código C ni acciones semánticas. Tampoco se pide diferenciar nivel léxico o sintáctico, sólo incluir las producciones necesarias para reconocer números reales (como los indicados como ejemplo).....	3
3.3 ¿Ves alguna diferencia, incluso sutil? ¿Los lenguajes generados o reconocidos por ambas representaciones difieren en algo? ¿Ves algún problema en implementar el analizador sintáctico, junto con su semántica siguiendo el diagrama?.....	4

3. Cuestiones de la Sesión 3

3.1 Adapta la gramática para que las expresiones encerradas con paréntesis puedan ir precedidas de un signo negativo (o positivo). Indicar sólo las producciones nuevas necesarias. Se recomienda probar la solución con bison. ¿Funciona para la expresión $-(-3)$?

Al igual que en la sesión anterior, hemos optado por generalizar el operando como una nueva expresión válida “resto_operando”, precedida de los signos, o no. De esta manera logramos obtener una solución de forma limpia y estructurada.

C/C++

```
operando:    resto_operando { $$ = $1; }
            | '-' resto_operando { $$ = -$2; }
            | '+' resto_operando { $$ = $2; } ;

resto_operando:
    NUMERO { $$ = $1; }
    | '(' expresion ')' { $$ = $2; } ;
```

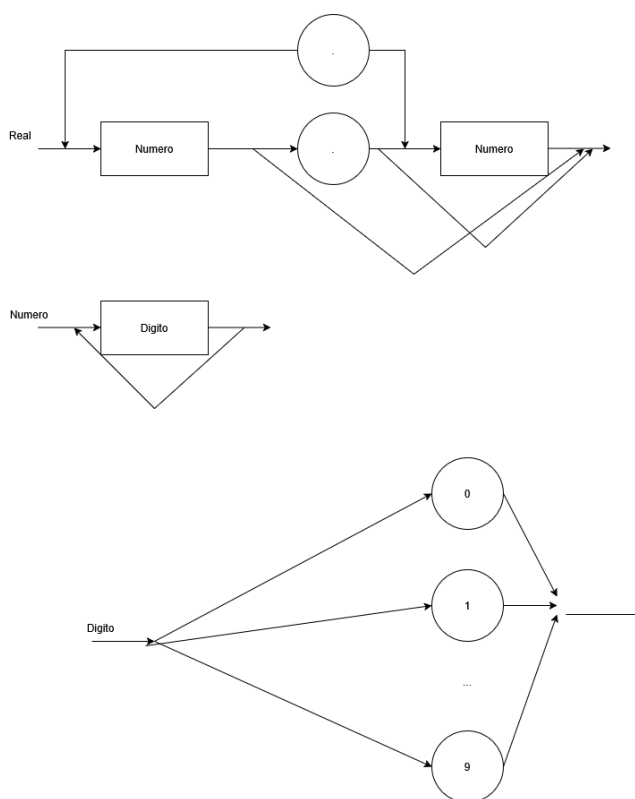
3.2 Adapta la gramática para reconocer números reales (ejemplos: 00.33, 0.33, .33, 100, 0., 100.33). Representa las nuevas producciones de la gramática en BNF y en forma de Diagrama Sintáctico. No se pide ni código C ni acciones semánticas. Tampoco se pide diferenciar nivel léxico o sintáctico, sólo incluir las producciones necesarias para reconocer números reales (como los indicados como ejemplo).

None

`real ::= NUMERO '.' | NUMERO '.' NUMERO | '.' NUMERO | NUMERO`

None

`[0-9]+.[0-9]* + .[0-9]+ + [0-9]+`



3.3 ¿Ves alguna diferencia, incluso sutil? ¿Los lenguajes generados o reconocidos por ambas representaciones difieren en algo? ¿Ves algún problema en implementar el analizador sintáctico, junto con su semántica siguiendo el diagrama?

Sí, en la representación de los diagramas los operadores se expresan mediante una caja llamada operador, mientras que en el fichero y los operadores son no terminales directamente, y están desglosadas cada una en una producción.

El único problema que podríamos encontrar es la complejidad de la gramática y una posible ambigüedad.

4. Cuestiones de la Sesión 4

4.1 Prueba las siguientes expresiones para comprobar el funcionamiento de la precedencia. ¿a qué conclusión llegas?

La conclusión a la que hemos podido llegar, consta en que no se respeta la prioridad de los operadores.

```
2*3+1
Expresion=8.000000
2+3*1
Expresion=5.000000
1+3*2
Expresion=7.000000
1*3+2
Expresion=5.000000
```

4.2 Reforma la gramática sustituyendo el no terminal operando por el de expresion en aquellas producciones que contengan una operación binaria. Recuerda que habíamos introducido operando para evitar la ambigüedad debida a la doble recursividad (expresionexpresion+expresion). Prueba de nuevo las expresiones anteriores.

Gracias a las declaraciones %left de la Sección 2, Bison ya sabe cómo resolver la ambigüedad (prioridad de operadores).

Cambios en calc4.y:

Fragmento de código

```
None
expresion: NUMERO                { $$ = $1; }
          | expresion '+' expresion { $$ = $1 + $3; }
          | expresion '-' expresion { $$ = $1 - $3; }
          | expresion '*' expresion { $$ = $1 * $3; }
          | expresion '/' expresion { $$ = $1 / $3; }
          | '(' expresion ')'      { $$ = $2; }
          | '-' expresion %prec SIGNO_UNARIO { $$ = -$2; }
          ;
```

Nota: Al usar %left, ya no necesitas el No Terminal operando para forzar el orden de evaluación.

4.3 Si nos plantearan usar variables como operandos dentro del lenguaje, ¿qué cambios plantearías en la gramática? Partimos de la idea de que las variables estarán identificadas por una secuencia alfanumérica, siendo el primer carácter una letra. No se contempla la definición previa de variables.

Añadimos un token denominado variable, al mismo tiempo que añadimos su expresión regular en el .lex.

4.4 Incluye la operación de asignación de una expresión a una variable en la gramática.

```
C/C++
%token VARIABLE
.
.
.
expresion:  NUMERO                { $$ = $1; }
           | VARIABLE              { $$ = 0; }
           | VARIABLE '=' expresion { $$ = $3; }
           | expresion '+' expresion { $$ = $1 + $3; }
           | expresion '-' expresion { $$ = $1 - $3; }
           | expresion '*' expresion { $$ = $1 * $3; }
           | expresion '/' expresion { $$ = $1 / $3; }
           | '(' expresion ')'      { $$ = $2; }
           | '-' expresion %prec SIGNO_UNARIO { $$ = -$2; }
           | '+' expresion %prec SIGNO_UNARIO { $$ = $2; }
           ;
```

4.5 Intenta adaptar el analizador lexicográfico correspondiente.

C/C++

```
[a-zA-Z_][a-zA-Z0-9_]*    { yylval = 0; return (VARIABLE); }
```

4.6 ¿Qué problema ves a la hora de implementar un analizador sintáctico que sea totalmente operativo (a nivel semántico) con las últimas cuestiones propuestas?

Al intentar implementar esto, surge un problema fundamental: ¿Dónde guardamos el valor de la variable?.

- Limitación actual: Bison solo maneja valores temporales en su pila (\$1, \$2, etc.).
+1
- Necesidad: Para que las variables sean operativas, necesitas una Tabla de Símbolos. Sin ella, el analizador reconoce que has escrito $x = 5$, pero no tiene memoria para "recordar" que x vale 5 cuando intentes hacer $x + 2$ más tarde